

FIELD GUIDE

티키타카 개발 노트

공부하고, 만들고, 확인하고, 다시 쓰는 개발 노트

학습 50 : 툴링 50

강한 개발자 + 강한 Agent = 더 강한 팀

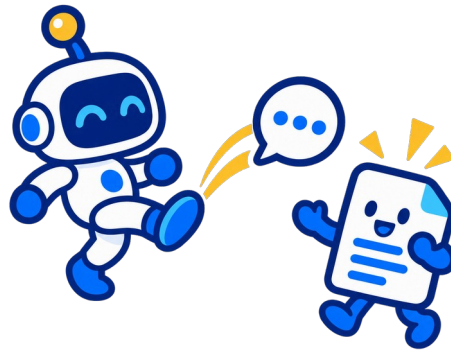
핵심 AI가 코드를 빨리 만들어도 마지막 판단은 사람이 한다. Agent는 대신 일하는 기계가 아니라, 같이 공부하고 확인하는 동료다.

프로젝트 소개 및 사용 가이드

티키타카 노트 · 데스크톱 학습·개발 노트 앱

PROJECT OVERVIEW

0. 프로젝트 한눈에 보기



한 줄 소개 티키타카 노트는 개발자가 Agent와 함께 공부하고 만든 내용을 한곳에 모아 두고, 다음 작업에서 다시 꺼내 쓰는 데스크톱 노트 앱이다.

누가 쓰면 좋은가

- AI 코딩 도구를 쓰지만, 만든 코드를 이해하고 지식도 남기고 싶은 개발자
- 풀스택이나 MES를 예제로 배우며 작은 기능부터 연습하고 싶은 입문자
- 프로젝트 규칙, 개발 패턴, 실패 경험을 팀 자산으로 쌓고 싶은 팀

무엇이 다른가

보통 노트는 적고 끝난다. 티키타카 노트는 아래 흐름을 계속 반복한다.

배운다 → 만든다 → 확인한다 → 적는다 → 다시 쓴다

지금 할 수 있는 일

- 카테고리와 주제 아래에 본문·하위 문서를 나누어 정리
- 글, 코드, 표를 편집하고 Agent에게 문서 추가·수정을 요청
- MES LOT 예제와 개발 노트를 보며 기능 단위로 학습
- SQLite에 로컬 저장하고 앱 안에서 백업·복원
- 데스크톱 앱 자동 업데이트와 릴리즈 설치 파일 제공

시작은 간단하다

1. 최신 릴리즈에서 내 운영체제용 설치 파일을 받는다.
2. 앱을 열고 들어 있는 샘플 노트를 하나 읽는다.
3. 작은 주제를 고른 뒤 직접 적거나 Agent에게 정리를 부탁한다.
4. 코드와 화면을 확인하고, 배운 점을 노트에 남긴다.

기술 구성 Next.js + Tauri + SQLite로 만든 로컬 우선 데스크톱 앱이다. 인터넷이 없어도 내 노트를 읽고 쓸 수 있고, 데이터는 내 컴퓨터에 저장된다.

현재 상태

- 현재 앱 버전: 0.1.58
- macOS와 Windows용 설치 패키지 제공
- 샘플 학습 노트와 기준 데이터를 설치 파일에 함께 포함
- 다음 단계: Agent 연동, 디자인 노트, 프롬프트·디버깅 지식 모음 강화

1. 왜 이 방식이 필요한가

AI로 기능을 빨리 만드는 일은 쉬워졌다. 하지만 사람이 결과를 이해하지 못하면 문제가 생긴다. 왜 이렇게 만들었는지 모르고, 틀린 코드가 쌓이고, 장애가 나도 원인을 찾기 어렵다.

메시지 빨리 만드는 것은 시작일 뿐이다. 작은 단위로 만들고, 사람이 확인하고, 배운 것을 적어야 실력이 남는다.

티키타카 노트는 이 과정을 한곳에서 이어 준다. 공부한 내용과 작업 결과를 남기고, 다음 개발에서 다시 꺼내 쓸 수 있게 만든다.

2. 핵심 공식: 학습 50 + 툴링 50



프로젝트 핵심 흐름: 기본기와 도구 활용을 5:5로 균형 있게 반복

학습은 기술 원리, 전체 구조, 코드 읽기, 문제 해결 능력을 키우는 일이다. 툴링은 Agent와 자동화를 써서 공부와 작업 속도를 높이는 일이다. 둘 중 하나만 잘해서는 오래가기 어렵다.

3. 티키타카의 다섯 가지 원칙

1. 기본기부터 쌓는다

프론트, 백엔드, DB, API를 전부 외울 필요는 없다. 대신 데이터가 어디서 와서 어디로 가는지는 설명할 수 있어야 한다.

바로 해보기 완료한 코드 하나를 골라 입력, 처리, 저장, 출력, 실패 흐름을 내 말로 설명한다.

2. Agent와 같이 공부한다

Agent에게 개념을 묻고 코드 검토도 부탁한다. 다만 복사하지 말고, 배운 내용을 짧게 다시 적는다.

바로 해보기 정답과 함께 왜 그런지, 다른 방법은 뭔지, 어떻게 확인할지도 물어본다.

3. 큰 기능을 한 번에 말기지 않는다

목표를 작은 구현 단위로 나누고 각 단위마다 확인한다. 범위가 작을수록 사람도 이해하기 쉽고 Agent도 맥락을 정확히 잡는다.

바로 해보기 한 번의 작업은 한 가지 동작과 하나의 완료 조건을 갖도록 쪼갬다.

4. 잘 만든 것은 팀의 자산으로 만든다

코드 규칙, API 방식, DB 설계, 오류 해결법, 실패한 시도를 다시 찾기 쉽게 남긴다.

바로 해보기 결과뿐 아니라 왜 골랐는지와 언제 다시 쓰면 되는지도 적는다.

5. 쌓은 지식을 다시 Agent에게 준다

팀의 규칙과 과거 경험을 Agent가 읽을 수 있게 만든다.

바로 해보기 노트를 작게 나누어 두고, 작업을 시작할 때 관련 자료만 Agent에게 준다.

4. 실행 루프: 공부에서 재사용까지

개발할 때 아래 일곱 단계를 차례로 돈다. 각 단계에서 짧은 기록을 남기면 다음 작업이 쉬워진다.

1. 목표 정의: 이번 작업이 해결할 문제와 완료 조건을 한 문장으로 적는다.
2. 기본기 확인: 관련 개념, 데이터 흐름, 실패 가능성을 먼저 훑는다.
3. 작은 단위 설계: API, 도메인, UI, 테스트를 한꺼번에 묶지 말고 검증 가능한 조각으로 나눈다.
4. Agent와 구현: 코드와 함께 다른 방법, 장단점, 예상 문제를 질문한다.
5. 사람이 검증: 코드를 읽고 실행하며 테스트, 로그, API 계약, 화면 동작을 확인한다.
6. 노트로 정리: 결정 이유, 검증 증거, 실패한 방법, 재사용 조건을 남긴다.
7. 다시 알려주기: 다음 작업에서 관련 노트를 Agent에게 주고, 새 결과를 또 기록한다.

루프의 완료 조건 코드가 동작하는 것만으로 끝내지 않는다. 사람이 설명할 수 있고, 검증 증거가 남아 있으며, 다음 사람이 다시 찾을 수 있어야 완료다.

5. 작은 작업으로 나누는 기준

작업을 작게 나누면 무엇이 바뀌었는지 알기 쉽고, 문제가 난 곳도 빨리 찾을 수 있다.

- 완료 조건이 하나인가: '검색된다', '저장된다', '권한이 거부된다'처럼 관찰 가능한 결과가 하나여야 한다.
- 변경 범위가 설명 가능한가: 무엇을 바꾸고 무엇을 바꾸지 않는지 명확해야 한다.
- 따로 확인할 수 있는가: 테스트, API 호출, 로그, 화면 중 하나로 바로 확인할 수 있어야 한다.

- 되돌리기 쉬운가: 실패해도 영향이 제한되고 이전 상태를 복구하기 쉬워야 한다.
- 지식이 남는가: 구현 뒤에 배운 점과 결정 이유를 짧게라도 기록할 수 있어야 한다.

6. 실전 예시: MES LOT 상태 필터

MES를 몰라도 괜찮다. 큰 기능을 작은 일로 나누는 흐름만 보면 된다. 'LOT 상태 필터를 전부 만들어줘'라고 한 번에 시키지 말고 아래처럼 나눈다.

1. 계약 먼저: status 쿼리의 허용 값, 기본값, 잘못된 값의 응답을 정의한다.
2. 백엔드 구현: 컨트롤러 파라미터, 조회 조건, 단위 또는 통합 테스트를 추가한다.
3. API 검증: OpenAPI 문서와 실제 요청 응답에서 파라미터가 노출되고 적용되는지 확인한다.
4. 프론트 구현: 필터 UI, URL 상태, API 호출 값을 연결한다.
5. 화면 검증: 선택값, 새로고침 유지, 빈 결과, 오류 상태를 확인한다.
6. 노트 작성: 계약, 구현 위치, 테스트 명령, 버전 순서와 실패 사례를 남긴다.

Agent에게 물을 질문 '이 구현에서 조용히 실패할 수 있는 지점은 무엇인가?', 'API 계약과 화면 상태가 어긋났음을 어떻게 검출할 수 있는가?', '테스트를 가장 작은 단위로 나누면 무엇인가?'

7. 다시 쓰기 좋은 노트의 기본 틀

노트는 길 필요가 없다. 아래 내용을 한 화면에서 찾을 수 있으면 사람도 Agent도 다시 쓰기 쉽다.

항목	반드시 남길 내용
문제	무엇이 필요했고 어떤 증상이 있었는가
상황	관련 모듈, 데이터 흐름, 제약 조건, 버전
결정	선택한 방법과 선택 이유
안 쓴 방법	검토했지만 쓰지 않은 방법과 이유
바꾼 곳	핵심 파일, API, 스키마, 코드 패턴
검증	실행한 테스트, 명령, 화면 확인, 로그
실패 기록	틀린 가정, 재현 조건, 해결 단서
다시 쓸 때	언제 다시 적용할 수 있고 언제 적용하면 안 되는가
태그	도메인, 기술, 장애 유형, 관련 기능

노트 기본 틀: 결과뿐 아니라 상황, 결정, 확인, 실패, 다시 쓸 조건까지 기록

8. Agent에게 줄 자료 묶음

문서를 전부 주면 Agent도 헛갈린다. 지금 작업에 필요한 자료만 골라서 준다.

- 프로젝트 규칙: 디렉터리 구조, 의존성 방향, 명명 규칙, 보안 금지사항
- 전체 구조 그림: 주요 모듈, 데이터 흐름, 외부 연동, 담당 범위
- 대표 패턴: 인증, 오류 처리, API 응답, DB 변경, 테스트 작성 예시
- 결정 기록: 기술을 고른 이유와 장단점, 바꾸기 어려운 조건
- 실패 지식: 반복되는 장애, 재현 절차, 확인할 로그와 지표

- 현재 상태: 진행 중 작업, 알려진 기술 부채, 다음 검증 포인트

운영 원칙 모든 문서를 한꺼번에 넣지 않는다. 현재 작업과 관련된 최소 문서만 선택하고, 오래된 정보에는 버전과 유효 기간을 표시한다.

9. 위험 신호와 대응

위험 신호	즉시 할 대응
코드는 많은데 설명할 수 없다	변경을 멈추고 데이터 흐름과 실패 경로를 직접 설명한다.
한 프롬프트가 여러 계층을 동시에 바꾼다	API, 도메인, 저장소, UI, 검증을 별도 작업으로 분리한다.
테스트가 Agent가 만든 코드만 따라간다	요구사항과 실패 조건에서 독립적으로 테스트 케이스를 다시 만든다.
노트가 결과 복사본에 그친다	결정 이유, 대안, 검증 증거, 재사용 조건을 추가한다.
과거 문서가 현재 코드와 어긋난다	문서에 기준 버전과 마지막 검증일을 적고 변경 시 함께 갱신한다.
Agent가 자신 있게 틀린다	근거 파일과 실행 결과를 요구하고, 불확실한 내용은 가정으로 표시한다.

10. 30일 도입 계획

1주차 · 한 기능을 작게

- 진행 중인 기능 하나를 선택한다.
- 작업을 3-5개 검증 가능한 단위로 나눈다.
- 각 단위에 완료 조건과 검증 방법을 적는다.

2주차 · 노트 형식을 고정

- 최소 노트 템플릿을 팀 저장소에 만든다.
- 완료한 작업 두 건을 템플릿으로 정리한다.
- 실패한 시도 한 건을 반드시 포함한다.

3주차 · Agent에 자료 연결

- 프로젝트 규칙과 대표 예시를 한 묶음으로 만든다.
- 작업 시작 시 관련 노트를 Agent에게 제공한다.
- 답변이 실제 규칙을 반영했는지 확인한다.

4주차 · 측정하고 다듬기

- 재작업, 검증 실패, 문서 재사용 사례를 모은다.
- 쓸모 없는 항목은 줄이고 빠진 검증 항목은 추가한다.
- 다음 달에 자동화할 한 가지를 고른다.

11. 측정 지표

Agent Engineering의 성과를 코드 생산량만으로 보면 잘못된 최적화를 하기 쉽다. 속도와 함께 이해, 검증, 재사용을 측정한다.

- **이해 가능성:** 변경자가 데이터 흐름과 설계 이유를 설명할 수 있는 비율
- **검증 통과율:** 첫 구현 뒤 따로 확인했을 때 통과한 작업의 비율
- **작은 배치 시간:** 하나의 완료 조건을 구현하고 검증하는 데 걸린 시간
- **지식 재사용률:** 새 작업에서 기존 노트나 패턴을 실제로 참조한 비율
- **결함 유출:** 검증 단계에서 잡히지 않고 다음 환경으로 넘어간 결함 수
- **최신 문서 비율:** 현재 코드와 내용이 맞는 핵심 문서의 비율

12. 제품 로드맵에 적용하기

현재 앱은 MES LOT 학습 노트와 Agent 기반 문서 관리를 제공한다. 다음 기능도 메뉴 수를 늘리기보다 '배우고 다시 쓰는 흐름'에 도움이 되는지 보고 고른다.

- **MCP를 통한 Agent 연동:** 필요한 노트를 작업 시점에 검색하고 공급
- **디자인 노트 관리:** UI 결정과 근거, 재사용 가능한 패턴 축적
- **프롬프트 라이브러리:** 좋았던 질문 구조와 검증 프롬프트 재사용
- **개발 패턴·코딩 컨벤션:** 팀의 기본 구현 방식을 명시
- **디버깅 지식 베이스:** 증상, 원인, 로그, 해결 순서를 검색 가능하게 저장
- **기술 부채 관리:** 임시 결정과 갚아야 할 조건을 추적
- **팀 지식 직접 참조:** Agent가 현재 규칙과 과거 결정을 근거로 답하도록 연결

13. Git 및 릴리즈 링크

아래 주소에서 소스 코드, 설치 파일, 변경 기록, 이슈를 볼 수 있다. 데스크톱 앱의 배포와 업데이트는 pkt-study-fullstack 저장소를 기준으로 한다.

Tauri 앱 소스: [dota-pilot1/pkt-study-fullstack](https://github.com/dota-pilot1/pkt-study-fullstack)

최신 릴리즈: [pkt-study-fullstack/releases/latest](https://github.com/pkt-study-fullstack/releases/latest)

전체 릴리즈: [pkt-study-fullstack/releases](https://github.com/pkt-study-fullstack/releases)

이슈 및 개선 제안: [pkt-study-fullstack/issues](https://github.com/pkt-study-fullstack/issues)

상위 샘플 프로젝트: [dota-pilot1/sample-pkt-project](https://github.com/dota-pilot1/sample-pkt-project)

설치 파일 받기 '최신 릴리즈'를 열고 내 운영체제에 맞는 설치 파일을 받는다. 버전과 게시일도 여기에서 확인할 수 있다.

학습 50 + 툴링 50 = Agent Engineering

사람이 공부하고, Agent와 만들고, 사람이 확인하고, 결과를 적는다. 그리고 다음 작업에서 다시 쓴다. 이 단순한 반복이 티키타카 노트의 핵심이다.